

Smart Contract Assessment

RAYLSFUSION LTD. (ParFin)

HALBORN

Smart Contract Assessment - RAYLSFUSION LTD. (ParFin)

Prepared by:  HALBORN

Last Updated 05/13/2026

Date of Engagement: March 2nd, 2026 - March 13th, 2026

Summary

100% ⓘ OF ALL REPORTED FINDINGS HAVE BEEN ADDRESSED

ALL FINDINGS	CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL
15	1	2	7	2	3

TABLE OF CONTENTS

1. Introduction	
2. Assessment summary	
3. Test approach & methodology	
4. Risk methodology	
5. Scope	
6. Assessment summary & findings overview	
7. Findings & Tech Details	
7.1 Pool reward balances permanently stranded due to missing claim integration in delegationpool	
7.2 Double-decrement in committee size calculation blocks valid slashing operations	
7.3 Slashed delegation wrls permanently locked in consensusregistry	
7.4 Distributeepochfees slippage protection disabled by 1:1 price assumption and spot oracle baseline	
7.5 Last validator permanently blocked from unstaking	
7.6 Unbonding delegation immune to slashing via requestundelegation front-running	
7.7 Non-validator delegators blocked from claiming stake rewards	
7.8 Rewarddistributor uses initial stake amount instead of post-slash balance for reward distribution	
7.9 Slash per-share rounding allows delegators to collectively overclaim rewards	
7.10 Entire active validator set can enter pendingexit simultaneously due to equality boundary in committee size check	
7.11 Arithmetic overflow in _eligibleforunstake for pendingexit validators	

7.12 Bls identity point detection unreachable due to input/constant size mismatch

7.13 Hardcoded erc-7201 storage slot does not match its documented namespace derivation

7.14 Ring buffer expansion causes permanent attestation dos

7.15 _eject() swap-and-pop breaks committee sort order after validator burn

1. INTRODUCTION

RAYLSFUSION engaged Halborn to perform a security assessment of their smart contracts from March 2nd, 2026 to March 16th, 2026 . The assessment scope was limited to the smart contracts provided to Halborn. Commit hashes and additional details are available in the Scope section of this report.

The Rayls smart contract system implements a proof-of-stake consensus layer where validators register with BLS keys and stake native RLS to join epoch-based committees, delegators pool ERC-20 RLS behind validators for proportional rewards and slashing, and protocol-collected transaction fees are swapped to RLS via a DEX and distributed across validators, delegators, treasury, and burn addresses, with cross-chain token bridging supported through LayerZero OFT and Axelar interchain adapters.

2. ASSESSMENT SUMMARY

Halborn assigned 1 full-time security engineer to conduct a comprehensive review of the smart contracts within scope. The engineer is an expert in blockchain and smart contract security, with advanced skills in penetration testing and smart contract exploitation, as well as extensive knowledge of multiple blockchain protocols.

The objectives of this assessment are to:

- Identify potential security vulnerabilities within the smart contracts.
- Verify that the smart contract functionality operates as intended.

In summary, **Halborn** identified several areas for improvement to reduce the likelihood and impact of risks, were mostly addressed by the **RAYLSFUSION team**. The main recommendations were:

- Round slashes in favor of the pool to prevent delegators from collectively withdrawing more rewards than were deposited.
- Track and enable recovery of slashed delegation tokens sent to the validator registry, which are currently permanently locked.
- Enforce the validator allowlist at every lifecycle transition, not just at initial registration, to prevent delisted validators from joining the active set.
- Use sort-preserving deletion when ejecting validators from committees to maintain the sorted order required by the protocol.
- Fill all new slots when expanding the attestation ring buffer to prevent out-of-bounds writes that permanently block future attestations.
- Fix the BLS identity point size constants so infinity detection actually triggers during point encoding.
- Restrict the BLS field encoding function to internal use or return modified data, since external callers silently receive unchanged results.
- Validate BLS key and signature lengths before forwarding to the cryptographic precompile to avoid opaque failures.
- Compute fee distribution slippage protection using correct token decimals and a time-weighted price rather than a manipulable spot price.

3. TEST APPROACH & METHODOLOGY

Halborn conducted a combination of manual code review and automated security testing to balance efficiency, timeliness, practicality, and accuracy within the scope of this assessment. While manual testing is crucial for identifying flaws in logic, processes, and implementation, automated testing enhances coverage of smart contracts and quickly detects deviations from established security best practices.

The following phases and associated tools were employed throughout the term of the assessment:

- Research into the platform's architecture, purpose and use.
- Manual code review and walkthrough of smart contracts to identify any logical issues.
- Comprehensive assessment of the safety and usage of critical Solidity variables and functions within scope that could lead to arithmetic-related vulnerabilities.
- Local testing using custom scripts.
- Fork testing against main networks.
- Static security analysis of scoped contracts, and imported functions (**Slither**).

4. RISK METHODOLOGY

Every vulnerability and issue observed by Halborn is ranked based on **two sets of Metrics** and a **Severity Coefficient**. This system is inspired by the industry standard Common Vulnerability Scoring System.

The two **Metric sets** are: **Exploitability** and **Impact**. **Exploitability** captures the ease and technical means by which vulnerabilities can be exploited and **Impact** describes the consequences of a successful exploit.

The **Severity Coefficients** is designed to further refine the accuracy of the ranking with two factors: **Reversibility** and **Scope**. These capture the impact of the vulnerability on the environment as well as the number of users and smart contracts affected.

The final score is a value between 0-10 rounded up to 1 decimal place and 10 corresponding to the highest security risk. This provides an objective and accurate rating of the severity of security vulnerabilities in smart contracts.

The system is designed to assist in identifying and prioritizing vulnerabilities based on their level of risk to address the most critical issues in a timely manner.

4.1 EXPLOITABILITY

ATTACK ORIGIN (AO):

Captures whether the attack requires compromising a specific account.

ATTACK COST (AC):

Captures the cost of exploiting the vulnerability incurred by the attacker relative to sending a single transaction on the relevant blockchain. Includes but is not limited to financial and computational cost.

ATTACK COMPLEXITY (AX):

Describes the conditions beyond the attacker’s control that must exist in order to exploit the vulnerability. Includes but is not limited to macro situation, available third-party liquidity and regulatory challenges.

METRICS:

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Origin (AO)	Arbitrary (AO:A) Specific (AO:S)	1 0.2
Attack Cost (AC)	Low (AC:L) Medium (AC:M) High (AC:H)	1 0.67 0.33

EXPLOITABILITY METRIC (M_E)	METRIC VALUE	NUMERICAL VALUE
Attack Complexity (AX)	Low (AX:L) Medium (AX:M) High (AX:H)	1 0.67 0.33

Exploitability E is calculated using the following formula:

$$E = \prod m_e$$

4.2 IMPACT

CONFIDENTIALITY (C):

Measures the impact to the confidentiality of the information resources managed by the contract due to a successfully exploited vulnerability. Confidentiality refers to limiting access to authorized users only.

INTEGRITY (I):

Measures the impact to integrity of a successfully exploited vulnerability. Integrity refers to the trustworthiness and veracity of data stored and/or processed on-chain. Integrity impact directly affecting Deposit or Yield records is excluded.

AVAILABILITY (A):

Measures the impact to the availability of the impacted component resulting from a successfully exploited vulnerability. This metric refers to smart contract features and functionality, not state. Availability impact directly affecting Deposit or Yield is excluded.

DEPOSIT (D):

Measures the impact to the deposits made to the contract by either users or owners.

YIELD (Y):

Measures the impact to the yield generated by the contract for either users or owners.

METRICS:

IMPACT METRIC (M_I)	METRIC VALUE	NUMERICAL VALUE
Confidentiality (C)	None (C:N) Low (C:L) Medium (C:M) High (C:H) Critical (C:C)	0 0.25 0.5 0.75 1

Impact Metric (M_I)	Metric Value	Numerical Value
Integrity (I)	None (I:N)	0
	Low (I:L)	0.25
	Medium (I:M)	0.5
	High (I:H)	0.75
	Critical (I:C)	1
Availability (A)	None (A:N)	0
	Low (A:L)	0.25
	Medium (A:M)	0.5
	High (A:H)	0.75
	Critical (A:C)	1
Deposit (D)	None (D:N)	0
	Low (D:L)	0.25
	Medium (D:M)	0.5
	High (D:H)	0.75
	Critical (D:C)	1
Yield (Y)	None (Y:N)	0
	Low (Y:L)	0.25
	Medium (Y:M)	0.5
	High (Y:H)	0.75
	Critical (Y:C)	1

Impact I is calculated using the following formula:

$$I = max(m_I) + \frac{\sum m_I - max(m_I)}{4}$$

4.3 SEVERITY COEFFICIENT

REVERSIBILITY (R):

Describes the share of the exploited vulnerability effects that can be reversed. For upgradeable contracts, assume the contract private key is available.

SCOPE (S):

Captures whether a vulnerability in one vulnerable contract impacts resources in other contracts.

METRICS:

Severity Coefficient (C)	Coefficient Value	Numerical Value
Reversibility (r)	None (R:N)	1
	Partial (R:P)	0.5
	Full (R:F)	0.25
Scope (s)	Changed (S:C)	1.25
	Unchanged (S:U)	1

Severity Coefficient *C* is obtained by the following product:

$$C = rs$$

The Vulnerability Severity Score *S* is obtained by:

$$S = \min(10, EIC * 10)$$

The score is rounded up to 1 decimal places.

SEVERITY	SCORE VALUE RANGE
Critical	9 - 10
High	7 - 8.9
Medium	4.5 - 6.9
Low	2 - 4.4
Informational	0 - 1.9

5. SCOPE

REPOSITORY

(a) Repository: [axyl](#)

(b) Assessed Commit ID: 26aeb3d

(c) Items in scope:

- `rayls-contracts/src/consensus/ConsensusRegistry.sol`
- `rayls-contracts/src/consensus/StakeManager.sol`
- `rayls-contracts/src/consensus/DelegationPool.sol`
- `rayls-contracts/src/consensus/Issuance.sol`
- `rayls-contracts/src/consensus/Blsg1.sol`
- `rayls-contracts/src/consensus/SystemCallable.sol`
- `rayls-contracts/src/fees/FeeAggregator.sol`
- `rayls-contracts/src/fees/RewardDistributor.sol`
- `rayls-contracts/src/recoverable-wrapper/RecoverableWrapper.sol`
- `rayls-contracts/src/recoverable-wrapper/RecordUtil.sol`
- `rayls-contracts/src/InterchainRLS.sol`
- `rayls-contracts/src/WRLS.sol`
- `rayls-contracts/src/faucet/RLFaucet.sol`
- `rayls-contracts/src/faucet/StablecoinManager.sol`
- `rayls-contracts/src/CI/GitAttestationRegistry.sol`
- `rayls-contracts/src/native/NativeTokenController.sol`
- `rayls-contracts/src/token/RLS.sol`

FILE

(a) Submitted File: `rayls-contracts-src.zip`

(b) Items in scope:

- `/WRLS.sol`
- `/SystemCallable.sol`
- `/StakeManager.sol`
- `/StablecoinManager.sol`
- `/RLFaucet.sol`
- `/RewardDistributor.sol`
- `/RecoverableWrapper.sol`
- `/RecordUtil.sol`
- `/Issuance.sol`
- `/InterchainRLS.sol`
- `/GitAttestationRegistry.sol`

- /FeeAggregator.sol
- /DelegationPool.sol
- /ConsensusRegistry.sol
- /BlsG1.sol

REMEDATION COMMIT ID:



- 38fba41
- 1660a43
- ee750a1
- c3ea85b
- 476ce95
- e4ea13d
- 60cfc50
- d8fe68a
- 6305f4e
- 47dd831
- 4995080
- 6e4c622

Out-of-Scope: New features/implementations after the remediation commit IDs.

6. ASSESSMENT SUMMARY & FINDINGS OVERVIEW



SECURITY ANALYSIS	RISK LEVEL	REMEDATION DATE
POOL REWARD BALANCES PERMANENTLY STRANDED DUE TO MISSING CLAIM INTEGRATION IN DELEGATIONPOOL	CRITICAL	SOLVED - 04/05/2026

SECURITY ANALYSIS	RISK LEVEL	REMEDATION DATE
DOUBLE-DECREMENT IN COMMITTEE SIZE CALCULATION BLOCKS VALID SLASHING OPERATIONS	HIGH	SOLVED - 04/05/2026
SLASHED DELEGATION WRLS PERMANENTLY LOCKED IN CONSENSUSREGISTRY	HIGH	SOLVED - 04/05/2026
DISTRIBUTEPOCHFEES SLIPPAGE PROTECTION DISABLED BY 1:1 PRICE ASSUMPTION AND SPOT ORACLE BASELINE	MEDIUM	SOLVED - 04/05/2026
LAST VALIDATOR PERMANENTLY BLOCKED FROM UNSTAKING	MEDIUM	SOLVED - 04/05/2026
UNBONDING DELEGATION IMMUNE TO SLASHING VIA REQUESTUNDELEGATION FRONT-RUNNING	MEDIUM	RISK ACCEPTED - 04/02/2026
NON-VALIDATOR DELEGATORS BLOCKED FROM CLAIMING STAKE REWARDS	MEDIUM	SOLVED - 04/05/2026
REWARDDDISTRIBUTOR USES INITIAL STAKE AMOUNT INSTEAD OF POST-SLASH BALANCE FOR REWARD DISTRIBUTION	MEDIUM	SOLVED - 04/05/2026
SLASH PER-SHARE ROUNDING ALLOWS DELEGATORS TO COLLECTIVELY OVERCLAIM REWARDS	MEDIUM	SOLVED - 04/05/2026
ENTIRE ACTIVE VALIDATOR SET CAN ENTER PENDINGEXIT SIMULTANEOUSLY DUE TO EQUALITY BOUNDARY IN COMMITTEE SIZE CHECK	MEDIUM	SOLVED - 04/06/2026
ARITHMETIC OVERFLOW IN _ELIGIBLEFORUNSTAKE FOR PENDINGEXIT VALIDATORS	LOW	SOLVED - 04/07/2026

SECURITY ANALYSIS	RISK LEVEL	REMEDIATION DATE
BLS IDENTITY POINT DETECTION UNREACHABLE DUE TO INPUT/CONSTANT SIZE MISMATCH	LOW	SOLVED - 04/07/2026
HARDCODED ERC-7201 STORAGE SLOT DOES NOT MATCH ITS DOCUMENTED NAMESPACE DERIVATION	INFORMATIONAL	SOLVED - 04/05/2026
RING BUFFER EXPANSION CAUSES PERMANENT ATTESTATION DOS	INFORMATIONAL	SOLVED - 03/24/2026
_EJECT() SWAP-AND-POP BREAKS COMMITTEE SORT ORDER AFTER VALIDATOR BURN	INFORMATIONAL	SOLVED - 04/07/2026

7. FINDINGS & TECH DETAILS

7.1 POOL REWARD BALANCES PERMANENTLY STRANDED DUE TO MISSING CLAIM INTEGRATION IN DELEGATIONPOOL

// CRITICAL

Description

The `ConsensusRegistry` contract accumulates native RLS rewards for delegation pools in a `poolRewardBalances` mapping during each epoch's `applyIncentives()` system call, but the corresponding `claimPoolRewards()` function is never invoked by any code path in `DelegationPool`. This results in a permanent and growing loss of native RLS consensus rewards for all validators with active delegation pools.

Each epoch, `applyIncentives()` splits validator rewards proportionally between `balances[validatorAddr]` (validator's own share) and `poolRewardBalances[validatorAddr]` (delegation pool's share). The `claimPoolRewards()` function restricts withdrawals to `msg.sender == delegationPool`, but `DelegationPool` never calls it and no other caller can satisfy this guard. The accumulated rewards are permanently inaccessible.

This is compounded by `_consensusBurn()`, which zeroes `balances[validatorAddress]` upon full slashing but leaves `poolRewardBalances[validatorAddress]` intact, orphaning those entries with no recovery path.

Code Location

`ConsensusRegistry.sol`: reward accumulation in `applyIncentives()`:

 Copy Code

```
144 | uint256 validatorShare = (rewardAmount * validatorStake) / totalStake;
145 | uint256 poolShare = rewardAmount - validatorShare;
146 |
147 | balances[validatorAddr] += validatorShare;
148 | poolRewardBalances[validatorAddr] += poolShare; // accumulated but never claimed
```

`ConsensusRegistry.sol`: unreachable withdrawal function:

 Copy Code

```
579 | function claimPoolRewards(address validatorAddress) external nonReentrant {
580 |     require(msg.sender == delegationPool, "Only DelegationPool");
581 |     uint256 amount = poolRewardBalances[validatorAddress];
582 |     require(amount > 0, "No pool rewards");
583 |     poolRewardBalances[validatorAddress] = 0;
584 |     Issuance(issuance).distributeStakeReward(delegationPool, amount);
585 | }
```

`ConsensusRegistry.sol`: `_consensusBurn()` omits `poolRewardBalances` cleanup:

 Copy Code

```
851 | balances[validatorAddress] = 0;
852 | // poolRewardBalances[validatorAddress] is never zeroed
```


Proof of Concept

In the following scenario, a validator with an active delegation pool accumulates consensus rewards over 5 epochs, but the delegation pool's share of those rewards is permanently locked because

`DelegationPool` never calls `claimPoolRewards()`

 Copy Code

```
function test_POC_poolRewardBalances_neverDrained() public {
    // Simulate 5 epochs of incentives
    RewardInfo[] memory rewards = new RewardInfo[](1);
    rewards[0] = RewardInfo(validator1, 100);

    for (uint256 epoch; epoch < 5; epoch++) {
        vm.prank(sysAddress);
        cr.applyIncentives(rewards);
    }

    // poolRewardBalances has accumulated significant rewards
    uint256 poolRewards = cr.poolRewardBalances(validator1);
    assertGt(poolRewards, 0, "poolRewardBalances should accumulate");

    // Attempting to call claimPoolRewards from any address other than delegationPool reverts
    vm.expectRevert("Only DelegationPool");
    cr.claimPoolRewards(validator1);

    // Pool rewards are permanently stranded
    assertGt(poolRewards, 0, "Stranded pool rewards confirmed");

    uint256 validatorBalance = cr.getRewards(validator1);
    emit log_named_uint("Stranded pool rewards (wei)", poolRewards);
    emit log_named_uint("Validator balance (wei)", validatorBalance);
    emit log_named_string("Status", "Pool rewards permanently locked in Issuance");
}
```

The test passes successfully, showing that ~28.6% of consensus rewards (36,865 RLS) are permanently stranded in `poolRewardBalances` while only the validator's own share (92,164 RLS) remains accessible. The `claimPoolRewards()` call reverts for any caller other than `DelegationPool`, which never invokes it.

 Copy Code

```
Ran 2 tests for test/audit/NemesisTest.t.sol:NM001_PoolRewardBalancesStrandedTest
[PASS] test_NM001_burn_doesNot_clear_poolRewardBalances() (gas: 155339)
[PASS] test_NM001_poolRewardBalances_neverDrained() (gas: 105181)
Logs:
  Stranded pool rewards (wei): 36865714285714285714290
  Validator balance (wei): 92164285714285714285710
  Status: Pool rewards permanently locked in Issuance

Suite result: ok. 2 passed; 0 failed; 0 skipped; finished in 20.89ms
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:H/Y:H (9.4)

Recommendation

Consider adding a claim integration in `DelegationPool` that withdraws accumulated pool rewards from `ConsensusRegistry`. `_consensusBurn()` should also zero `poolRewardBalances` to prevent orphaned entries.

Remediation Comment

SOLVED: The **Rayls** team fully solved this finding in the specified commit by rebuilding the delegation reward path. Rewards now flow directly from **RewardDistributor** to **DelegationPool**, bypassing the broken **ConsensusRegistry** intermediary entirely. The **poolRewardBalances** mapping is no longer written or read anywhere, so nothing can get stranded.

Remediation Hash

<https://github.com/raylsnetwork/axyl/commit/38fba4187e2bd5154f1fc10b14d554b8da4cd654>

7.2 DOUBLE-DECREMENT IN COMMITTEE SIZE CALCULATION BLOCKS VALID SLASHING OPERATIONS

// HIGH

Description

The `ejectFromCommittees()` function in `ConsensusRegistry` miscalculates the post-ejection committee size due to a double-decrement, causing `checkCommitteeSize()` to revert on legitimate slashing operations.

When a validator's stake is fully slashed to zero, `consensusBurn()` calls `ejectFromCommittees()` to remove the validator from all three committee windows (current, next, and subsequent epoch). After each ejection, the function computes the resulting committee size and passes it to `_checkCommitteeSize()` to ensure the committee is not left empty.

The issue is that `eject()` operates directly on a storage array and calls `committee.pop()`, which already decrements `committee.length` in place. When `ejectFromCommittees()` subsequently reads `currentCommittee.length`, it gets the already-decremented value, then subtracts 1 again via the ternary expression. The computed `committeeSize` is therefore one less than the actual post-ejection committee size.

Code Location

`ConsensusRegistry.sol` double-decrement after ejection:

 Copy Code

```
786 | bool ejected = _eject(currentCommittee, validatorAddress);
787 | uint256 committeeSize = ejected
788 |   ? currentCommittee.length - 1 // ← reads already-decremented length, subtracts again
789 |   : currentCommittee.length;
790 | _checkCommitteeSize(numEligible, committeeSize);
```

`ConsensusRegistry.sol` `_eject()` modifies storage array in place:

 Copy Code

```
817 | function _eject(
818 |     address[] storage committee,
819 |     address validatorAddress
820 | ) internal returns (bool) {
821 |     uint256 len = committee.length;
822 |     for (uint256 i; i < len; ++i) {
823 |         if (committee[i] == validatorAddress) {
824 |             committee[i] = committee[len - 1];
825 |             committee.pop(); // ← already decrements committee.length
826 |             return true;
827 |         }
828 |     }
829 |     return false;
830 | }
```

Proof of Concept

In the following scenario, a validator is slashed to zero stake but the slash reverts because the double-decrement incorrectly computes the committee size as zero.

 Copy Code

```
function test_POC_ejectFromCommittees_doubleDecrement() public {
    // Setup: 2-member committee, slash validator1 to zero
    Slash[] memory slashes = new Slash[](1);
    slashes[0] = Slash(validator1, stakeAmount_ * 2); // slash exceeds balance

    vm.prank(sysAddress);
    // Reverts with InvalidCommitteeSize because committeeSize is
    // computed as 0 (double-decremented) instead of 1 (actual)
    vm.expectRevert();
    cr.applySlashes(slashes);

    // Validator remains in committee with zero economic backing
    assertGt(cr.poolRewardBalances(validator1), 0);
}
```

The test passes successfully, showing that the slash reverts with `InvalidCommitteeSize` despite the committee still having one valid member after ejection.

 Copy Code

```
[PASS] test_NM004_ejectFromCommittees_doubleDecrement()
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:H/I:M/D:N/Y:N (8.8)

Recommendation

Consider using `currentCommittee.length` directly after `eject()` returns, as the storage array is already decremented by `committee.pop()`. The same correction applies to all three ejection points in `ejectFromCommittees()`.

Remediation Comment

SOLVED: The `Rayls` team solved this finding in the specified commit by removing the ternary double-decrement and passing `committee.length` directly to `checkCommitteeSize()` after each `eject()` call, since `_eject()` already decrements the storage array length via `pop()` in place.

Remediation Hash

<https://github.com/raylsnetwork/axyl/commit/1660a4303acd98e040c423f6220fe63f07b9bcb7>

7.3 SLASHED DELEGATION WRLS PERMANENTLY LOCKED IN CONSENSUSREGISTRY

// HIGH

Description

The `DelegationPool.applyPoolSlash()` function transfers slashed ERC-20 wRLS tokens directly to ConsensusRegistry. The ConsensusRegistry contract has no ERC-20 accounting and no ERC-20 recovery mechanism. Its `receive()` function only handles native ETH (incrementing the `slashedFunds` counter), and `withdrawSlashedFunds()` withdraws only from that native-ETH counter. The slashed wRLS tokens arrive in ConsensusRegistry's ERC-20 balance with no corresponding ledger entry and no withdrawal path, making them permanently unrecoverable.

Every slashing event affecting a validator with active delegators results in permanent, irrecoverable loss of wRLS tokens equal to the pool's proportional slash share. Governance's intended recovery path via `withdrawSlashedFunds()` silently receives nothing from delegation slashes, and the locked amount grows proportionally with delegated stake and slash magnitude.

Code Location

 Copy Code

```
// DelegationPool.sol L518-548
function applyPoolSlash(
    address validatorAddress,
    uint256 amount
) external override onlyConsensusRegistry {
    if (amount == 0) return;
    if (!poolRegistered[validatorAddress]) return;

    ValidatorPool storage pool = _validatorPools[validatorAddress];

    if (pool.totalDelegated == 0) {
        emit PoolSlashed(validatorAddress, 0);
        return;
    }

    uint256 effectiveSlash = amount > pool.totalDelegated
        ? pool.totalDelegated
        : amount;

    pool.slashPerShareAccum +=
        (effectiveSlash * PRECISION) /
        pool.totalDelegated;

    pool.totalDelegated -= effectiveSlash;

    // @audit transfers ERC-20 wRLS to ConsensusRegistry, which has no ERC-20 tracking
    _rls.safeTransfer(address(consensusRegistry), effectiveSlash);

    emit PoolSlashed(validatorAddress, effectiveSlash);
}

// ConsensusRegistry.sol L591-595
// @audit receive() only handles native ETH — ERC-20 transfers never invoke this
receive() external payable {
    require(msg.sender == delegationPool, "Only DelegationPool");
    slashedFunds += msg.value;
}

// ConsensusRegistry.sol L600-607
// @audit withdrawSlashedFunds() operates on slashedFunds (native ETH) only — no ERC-20 path
function withdrawSlashedFunds(address to, uint256 amount) external onlyOwner {
```

Recommendation

Consider unwrapping wRLS to native RLS before transfer. This allows the existing `receive()` to `slashedFunds` to `withdrawSlashedFunds()` flow to function as designed without changes to ConsensusRegistry.

Remediation Comment

SOLVED: The `Rayls` team fully solved this finding in specified commit as part of a broader native-to-ERC-20 pivot for consensus staking. `applyPoolSlash` now returns the slashed amount, which `ConsensusRegistry.applySlashes` records into the `slashedFunds` counter. The obsolete native-ETH `receive()` function was deleted, and `withdrawSlashedFunds` now drains the counter via ERC-20 transfer. Slashed delegator wRLS is now properly accounted and fully recoverable by governance.

Remediation Hash

<https://github.com/raylsnetwork/axyl/commit/38fba4187e2bd5154f1fc10b14d554b8da4cd654>

7.4 DISTRIBUTE EPOCH FEES SLIPPAGE PROTECTION DISABLED BY 1:1 PRICE ASSUMPTION AND SPOT ORACLE BASELINE

// MEDIUM

Description

`FeeAggregator.distributeEpochFees()` swaps accumulated USDr fees to RLS via an Algebra DEX pool with two slippage guards, both broken.

The `minRlsOut` floor assumes $1 \text{ USDr} = 1 \text{ RLS}$. If RLS is more valuable, the floor exceeds fair output and every swap reverts (permanent DoS). If RLS is less valuable, the floor sits below fair output and offers no protection. A source comment acknowledges this is not production-ready. The `limitSqrtPrice` guard reads the pool's instantaneous spot price via `globalState()`. A same-block front-running trade shifts the spot price before the system call reads it, so the derived limit validates the very manipulation it was meant to prevent.

Neither protection layer functions as intended. The combined effect is that the protocol's epoch fee distribution has no effective slippage protection. An attacker can sandwich the `distributeEpochFees` system call, front-running with a large trade to shift the pool price, then back-running to restore it extracting value from every epoch's fee swap. The protocol receives significantly fewer RLS tokens than fair market value for the swapped USDr. Additionally, if the RLS/USDr exchange rate diverges from 1:1 with RLS being more valuable, the `minRlsOut` floor causes every swap to revert, permanently blocking fee distribution until governance intervenes.

Code Location

FeeAggregator.sol: `minRlsOut` calculated from 1:1 assumption:

```
327 | uint256 minRlsOut = (amountToProcess * (MAX_BPS - $.maxSlippageBps)) / MAX_BPS;
```

 Copy Code

FeeAggregator.sol: `limitSqrtPrice` derived from manipulable spot price:

```
380 | IAlgebraPool pool = IAlgebraPool(poolConfig.pool);
381 | (uint160 sqrtPriceX96,,,,) = pool.globalState();
382 |
383 | uint160 limitSqrtPrice;
384 | if (poolConfig.zeroForOne) {
385 |     limitSqrtPrice = uint160((uint256(sqrtPriceX96) * (MAX_BPS - $.maxSlippageBps)) / MAX_BPS)
386 |     if (limitSqrtPrice == 0) limitSqrtPrice = 1;
387 | } else {
388 |     limitSqrtPrice = uint160((uint256(sqrtPriceX96) * (MAX_BPS + $.maxSlippageBps)) / MAX_BPS)
389 | }
```

 Copy Code

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:M/I:M/D:N/Y:N (6.3)

Recommendation

Consider deriving `minRlsOut` from a proper price estimate (query the pool, use a cached price, or accept an explicit minimum from the caller) rather than assuming 1:1 parity.

Remediation Comment

SOLVED: The Rayls team solved this finding in the specified commit by replacing the broken on-chain slippage math with a caller-provided `minRlsOut` that is enforced post-swap, closing the sandwich attack vector.

Remediation Hash

<https://github.com/raylsnetwork/axyl/commit/ee750a171e0e074f62506c356189787a527a565e>

7.5 LAST VALIDATOR PERMANENTLY BLOCKED FROM UNSTAKING

// MEDIUM

Description

The `unstake()` function in `StakeManager` calls `removeValidator()` to delete the validator from the `validatorsAddresses` array before checking whether the array is now empty. When the caller is the sole remaining validator, the removal reduces the length from 1 to 0, and the subsequent if (`validatorsAddresses.length == 0`) revert `InvalidValidatorSupply()` check unconditionally reverts the transaction. Because EVM transactions are atomic, the revert also rolls back the preceding `_retire()` call in `ConsensusRegistry.unstake()`, leaving the validator permanently stuck, eligible to unstake but unable to complete it. No administrative function exists to recover the locked stake.

The last validator in the system permanently loses access to their staked funds. Their status cannot transition to retired, leaving a dangling validator entry that satisfies exit criteria but can never complete the unstaking lifecycle. While this is unlikely during normal operation with an active validator set, it becomes plausible during network wind-down, migration to a new contract version, or extended periods of low participation.

Code Location

`StakeManager.sol` : The internal `_unstake()` function removes the validator before checking the array length, causing the check to evaluate post-removal state.

 Copy Code

```
136 function _unstake(address validatorAddress, address recipient) internal virtual returns (uint256) {
137     _removeValidator(validatorAddress); // array shortened first
138     if (validatorsAddresses.length == 0) revert InvalidValidatorSupply(); // check evaluates post-removal
139     ...
140 }
```

`ConsensusRegistry.sol` : The public `unstake()` entry point calls `retire()` before `unstake()`, so the revert rolls back the retirement as well.

 Copy Code

```
546 function unstake(address validatorAddress) external override whenNotPaused nonReentrant {
547     address recipient = _getRecipient(validatorAddress);
548     if (msg.sender != validatorAddress && msg.sender != recipient)
549         revert NotRecipient(recipient);
550
551     ValidatorInfo storage validator = validators[validatorAddress];
552     if (!_eligibleForUnstake(validator)) revert IneligibleUnstake(validator);
553
554     _retire(validator); // rolled back by the revert below
555     uint256 stakeAndRewards = _unstake(validatorAddress, recipient); // reverts here
556     emit RewardsClaimed(recipient, stakeAndRewards);
557 }
```

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:M/I:N/D:H/Y:N (5.9)

Recommendation

Consider moving the minimum-supply check before the removal so it evaluates the pre-removal length. Checking `validatorsAddresses.length <= 1` before calling `_removeValidator()` preserves the same invariant without the order-of-operations conflict, requiring only a single-line reordering with no impact on downstream logic.

Remediation Comment

SOLVED: The `Rayls` team solved this finding in the specified commit by moving the `InvalidValidatorSupply` guard before `_removeValidator()` and changing the condition from `== 0` to `<= 1`, ensuring the revert fires before any state mutation occurs.

Remediation Hash

<https://github.com/raylsnetwork/axyl/commit/c3ea85b51a0182df4ab08a1696a2f4631caf0acd>

7.6 UNBONDING DELEGATION IMMUNE TO SLASHING VIA REQUESTUNDELEGATION FRONT-RUNNING

// MEDIUM

Description

`DelegationPool.requestUndelegation()` immediately subtracts the undelegated amount from `pool.totalDelegated` and stores it in `pos.undelegateAmount`. When `applyPoolSlash()` is subsequently called by the system at the epoch boundary, it distributes the slash proportionally across `pool.totalDelegated` which no longer includes the unbonding stake. A delegator who observes imminent validator misbehavior can call `requestUndelegation()` before the slash is applied, moving their tokens out of the slash base entirely.

`completeUndelegation()` then returns the full `pos.undelegateAmount` to the delegator with no retrospective slash check. The unbonding tokens are never subject to any penalty. If all delegators undelegated before the slash, `pool.totalDelegated` reaches zero and `applyPoolSlash()` returns early and the slash is entirely nullified.

Code Location

DelegationPool.sol : `requestUndelegation()` immediately reduces the slash base:

```
387 pos.undelegateAmount = amount;
388
389 uint32 currentEpoch = consensusRegistry.getCurrentEpoch();
390 pos.undelegateEpoch = uint64(currentEpoch) + config.unbondingEpochs;
391
392 // reduce pool total
393 pool.totalDelegated -= amount;
```

 Copy Code

DelegationPool.sol : `applyPoolSlash()` only slashes the (now-reduced) `pool.totalDelegated`:

```
527 if (pool.totalDelegated == 0) {
528     emit PoolSlashed(validatorAddress, 0);
529     return;
530 }
531
532 uint256 effectiveSlash = amount > pool.totalDelegated
533     ? pool.totalDelegated
534     : amount;
535
536 pool.slashPerShareAccum +=
537     (effectiveSlash * PRECISION) /
538     pool.totalDelegated;
```

 Copy Code

DelegationPool.sol : `completeUndelegation()` returns the full amount with no slash deduction:

```
414 uint256 amount = pos.undelegateAmount;
415
416 pos.undelegateEpoch = 0;
417 pos.undelegateAmount = 0;
418
419 if (amount > 0) {
420     _rls.safeTransfer(msg.sender, amount);
421 }
```

 Copy Code

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:M/Y:N (5.0)

Recommendation

Include unbonding stake in the slash base by either keeping `pool.totalDelegated` unchanged until `completeUndelegation()`, or by snapshotting `slashPerShareAccum` at undelegation time and applying any accumulated slash delta to `pos.undelegateAmount` during `completeUndelegation()`.

Remediation Comment

RISK ACCEPTED: The `Rayls` team made a business decision to accept the risk of this finding and not alter the contracts, stating that:

Accepted risk — by design. Unbonding stake is intentionally excluded from the slash base upon `requestUndelegation()`. The unbonding period (`config.unbondingEpochs`) serves as the protection window. This is a deliberate design choice documented in the contract NatSpec, consistent with other PoS systems (e.g., Ethereum's approach where withdrawals in progress are not retroactively slashed). Front-running is mitigated by the unbonding delay.

7.7 NON-VALIDATOR DELEGATORS BLOCKED FROM CLAIMING STAKE REWARDS

// MEDIUM

Description

`claimStakeRewards()` enforces a validator-index lookup on `msg.sender` as its first guard, before determining whether the caller is the designated reward recipient. When a delegation exists, `_getRecipient()` returns the delegator's address, making them the intended claimant. However, a delegator who is not themselves a registered validator fails the `validatorIndex[msg.sender] == 0` check and is rejected with `ValidatorNotFound` before the recipient check is ever reached. The delegator has no on-chain path to independently claim rewards they are entitled to and must rely entirely on the validator to call `claimStakeRewards` on their behalf.

Code Location

ConsensusRegistry.sol : validator-index guard fires before recipient check

 Copy Code

```
503 function claimStakeRewards(  
504     address validatorAddress  
505 ) external override whenNotPaused nonReentrant {  
506     if (validatorIndex[msg.sender] == 0) revert ValidatorNotFound(msg.sender); // ← fires first  
507  
508     uint8 validatorVersion = validators[validatorAddress].stakeVersion;  
509  
510     // require caller is either the validator or its delegator  
511     address recipient = _getRecipient(validatorAddress);  
512     if (msg.sender != validatorAddress && msg.sender != recipient)  
513         revert NotRecipient(recipient);  
514     uint256 rewards = _claimStakeRewards(  
515         validatorAddress,  
516         recipient,  
517         validatorVersion  
518     );  
519  
520     emit RewardsClaimed(recipient, rewards);  
521 }
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:M/I:N/D:N/Y:N (5.0)

Recommendation

Consider moving the recipient derivation before the validator-index guard, and apply the index check against the `validatorAddress` parameter rather than `msg.sender`, so that the delegator is recognized as a valid caller.

Remediation Comment

SOLVED: The `Rayls` team solved this finding in the specified commit by changing the guard in `claimStakeRewards()` from `validatorIndex[msg.sender]` to

`validatorIndex[validatorAddress]`, unblocking non-validator delegators from claiming their entitled rewards.

Remediation Hash

<https://github.com/raylsnetwork/axyl/commit/476ce95a1dcad77ad0c8788a68f6eed7aaff54f6>

7.8 REWARDDISTRIBUTOR USES INITIAL STAKE AMOUNT INSTEAD OF POST-SLASH BALANCE FOR REWARD DISTRIBUTION

// MEDIUM

Description

`RewardDistributor.distributeRewards()` calculates each validator's share of ERC-20 fee rewards proportional to their stake. It fetches stake data via `getBalanceBreakdown()`, which returns three values in order: (outstandingBalance, initialStakeAmount, rewards). The destructuring captures the second value `initialStakeAmount` which is the fixed amount from the stake configuration at the time the validator originally staked. After a slash reduces a validator's actual on-chain balance, `initialStakeAmount` remains unchanged. The slashed validator continues receiving reward shares proportional to their full original stake rather than their reduced actual balance.

Slashed validators receive a disproportionately large share of ERC-20 fee-based rewards. Healthy validators are under-rewarded by the corresponding amount. The asymmetry compounds across epochs.

Code Location

RewardDistributor.sol : captures `initialStakeAmount` (second return value) instead of `outstandingBalance` (first):

 Copy Code

```
122 | (, uint256 validatorStake, ) = IStakeManager(address(_consensusRegistry)).getBalanceBreakdown(
```

ConsensusRegistry.sol : `getBalanceBreakdown` return order confirms `outstandingBalance` is first:

 Copy Code

```
302 | function getBalanceBreakdown(address validatorAddress) public view override returns (uint256,
303 |     uint8 validatorVersion = validators[validatorAddress].stakeVersion;
304 |     uint256 initialStakeAmount = versions[validatorVersion].stakeAmount;
305 |     uint256 rewards = _getRewards(validatorAddress, initialStakeAmount);
306 |     uint256 outstandingBalance = balances[validatorAddress];
307 |
308 |     return (outstandingBalance, initialStakeAmount, rewards);
309 | }
```

RewardDistributor.sol : the fetched stake flows into proportional reward distribution:

 Copy Code

```
277 | uint256 validatorReward = (totalRewards * totalValidatorStake) / totalStake;
```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:M (5.0)

Recommendation

Consider using the first return value (`outstandingBalance`) instead of the second (`initialStakeAmount`) when destructuring the `getBalanceBreakdown` call, so reward distribution

reflects each validator's actual remaining stake after slashes.

Remediation Comment

SOLVED: The `Rayls` team solved this finding in specified commit by changing all `getBalanceBreakdown` destructuring in `RewardDistributor.sol` from position [1] (`initialStakeAmount`) to position [0] (`outstandingBalance`), correctly reflecting post-slash balances in reward distribution.

Remediation Hash

<https://github.com/raylsnetwork/axyl/commit/e4ea13dd09684973e1a2abf9280ab9b9dfa86c96>

7.9 SLASH PER-SHARE ROUNDING ALLOWS DELEGATORS TO COLLECTIVELY OVERCLAIM REWARDS

// MEDIUM

Description

The `DelegationPool` contract uses a MasterChef-style per-share accumulator to distribute slashes across delegators. Two sequential floor-division operations, one when updating the global `slashPerShareAccum` in `applyPoolSlash`, and another when settling each delegator's individual slash in `_settlePosition`, compound a rounding deficit. The truncated accumulator increment means the total recorded slash is slightly less than the actual `effectiveSlash`. Then, when each delegator's position is settled, a second floor division further reduces their individual slash amount. The result is that the sum of all individual slashes across delegators is strictly less than the `effectiveSlash` that was applied to `totalDelegated`. Delegator positions collectively represent more stake than `totalDelegated` reflects. When rewards are subsequently injected via `distributePoolRewards`, the per-share reward is calculated against the reduced `totalDelegated`, but the under-slashed positions claim against their inflated amounts, draining the pool beyond what was deposited and rendering it insolvent.

When the pool becomes insolvent, the last delegator(s) to claim rewards receive less than their entitlement, or the pool's ERC-20 balance is exhausted before all claims are fulfilled. The rounding error accumulates across every slash-then-reward cycle and scales with pool size and number of delegators.

Code Location

DelegationPool.sol : `applyPoolSlash` : The accumulator increment uses floor division, losing dust.

```
538 | pool.slashPerShareAccum +=
539 |     (effectiveSlash * PRECISION) /
540 |     pool.totalDelegated;           // floor division: dust lost here
```

 Copy Code

DelegationPool.sol : `applyPoolSlash` : `totalDelegated` is reduced by the full `effectiveSlash`, not the recoverable amount.

```
543 | pool.totalDelegated -= effectiveSlash;
```

 Copy Code

DelegationPool.sol : `_settlePosition` : Each delegator's individual slash is also truncated by a second floor division.

```
661 | uint256 accumulatedSlash = (pos.amount * pool.slashPerShareAccum) /
662 |     PRECISION;           // second floor division per delegator
663 | uint256 slashAmount = accumulatedSlash - pos.slashDebt;
```

 Copy Code

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:N/D:N/Y:M (5.0)

Recommendation

Consider using ceiling division ("round-in-favour-of-pool") when computing each delegator's individual slash in `_settlePosition`, ensuring the sum of individual slashes never falls below `effectiveSlash` and maintaining pool solvency across all slash-then-reward cycles.

Remediation Comment

SOLVED: The `Rayls` team fixed this finding in the specified commit with a coordinated rounding patch. `applyPoolSlash` only transfers the portion of the slash it can faithfully account for, leaving any rounding dust in the pool as a safety buffer, while all delegator-side slash calculations now use ceiling division to ensure each delegator absorbs at least their fair share.

Remediation Hash

<https://github.com/raylsnetwork/axyl/commit/60cfc5088cfe9ca15a1cbd0a54b9935430a1924b>

7.10 ENTIRE ACTIVE VALIDATOR SET CAN ENTER PENDINGEXIT SIMULTANEOUSLY DUE TO EQUALITY BOUNDARY IN COMMITTEE SIZE CHECK

// MEDIUM

Description

The `_checkCommitteeSize()` function in `ConsensusRegistry` uses a strict inequality (`committeeSize > activeOrPending`) that permits `beginExit()` to succeed when the number of active validators equals the committee size. This allows every active validator to enter `PendingExit` in the same epoch, potentially leaving the next epoch with zero eligible validators if the off-chain protocol layer fails to block it.

The protocol's `concludeEpoch` system call is then responsible for deciding which exits to process, but no on-chain constraint prevents it from processing all of them if the off-chain logic has a bug or misconfiguration. A coordinated or accidental mass exit would leave the network with zero active validators for the next epoch, halting consensus.

Code Location

`ConsensusRegistry.sol` : `beginExit()` function queries the active validator count and current committee size, then delegates to `_checkCommitteeSize()` to enforce a minimum.

 Copy Code

```
524 function beginExit() external override whenNotPaused {
525     if (validatorIndex[msg.sender] == 0) revert ValidatorNotFound(msg.sender);
526
527     // disallow filling up the exit queue
528     uint256 numActive = _getValidators(ValidatorStatus.Active).length;
529     uint256 committeeSize = epochInfo[epochPointer].committee.length;
530     _checkCommitteeSize(numActive, committeeSize);
531
532     // require caller status is `Active` and `currentEpoch >= activationEpoch`
533     _checkValidatorStatus(msg.sender, ValidatorStatus.Active);
534     ValidatorInfo storage validator = validators[msg.sender];
535     uint32 current = currentEpoch;
536     if (current < validator.activationEpoch) {
537         revert InvalidEpoch(current);
538     }
539
540     // enter validator in pending exit queue
541     _beginExit(validator);
542 }
```

`ConsensusRegistry.sol` : The guard itself only reverts when the committee would strictly exceed the active count, allowing the equality case to pass.

 Copy Code

```
function _checkCommitteeSize(
    uint256 activeOrPending,
    uint256 committeeSize
) internal pure {
    if (
        activeOrPending == 0 ||
        committeeSize == 0 ||
        committeeSize > activeOrPending // ← strict inequality: equality passes
    ) {
```

```
    revert InvalidCommitteeSize(activeOrPending, committeeSize);  
  }  
}
```

ConsensusRegistry.sol : each successful **beginExit()** transitions the caller to **PendingExit**, which **_getValidators(Active)** still includes in its count (Active, PendingActivation, and PendingExit are all considered "active" for committee eligibility). This means **numActive** does not decrease after a **beginExit()** call within the same epoch, so the check continues to pass for every subsequent validator.

 Copy Code

```
703 | function _beginExit(ValidatorInfo storage validator) internal {  
704 |     validator.currentStatus = ValidatorStatus.PendingExit;  
705 |     validator.exitEpoch = PENDING_EPOCH;  
706 |  
707 |     emit ValidatorPendingExit(validator);  
708 | }
```

BVSS

AO:A/AC:L/AX:M/R:N/S:U/C:N/A:H/I:N/D:N/Y:N (5.0)

Recommendation

Consider changing the strict inequality to a non-strict one so that at least one active validator must always remain outside the exit queue.

Remediation Comment

SOLVED: The **Rayls** team solved this finding in the specified commit by adding a **numStrictlyActive** computation in **beginExit()** that subtracts **PendingExit** validators from the eligible count, paired with a **<= 1** guard that blocks the last strictly-Active validator from entering the exit queue.

Remediation Hash

<https://github.com/raylsnetwork/axyl/commit/476ce95a1dcad77ad0c8788a68f6eed7aaff54f6>

7.11 ARITHMETIC OVERFLOW IN `_ELIGIBLEFORUNSTAKE` FOR `PENDINGEXIT` VALIDATORS

// LOW

Description

`eligibleForUnstake()` computes `validator.exitEpoch + 1` unconditionally for every non-Staked validator status, before checking whether the status is actually Exited. When `beginExit()` is called, `exitEpoch` is set to `PENDINGEPOCH = type(uint32).max`. Any call to `unstake()` while the validator remains in `PendingExit` status evaluates `type(uint32).max + 1`, which triggers an arithmetic overflow panic (0x11) under Solidity 0.8's checked arithmetic. The caller receives a raw EVM panic instead of the expected `IneligibleUnstake` custom error.

The validator or their delegator calling `unstake()` during `PendingExit` receives an opaque `Panic(0x11)` instead of the descriptive `IneligibleUnstake` error. Off-chain tooling that catches `IneligibleUnstake` to display "wait for exit finalization" will miss this case and surface a generic failure.

Code Location

ConsensusRegistry.sol : `exitEpoch + 1` executes before the Exited status check:

```
976 function _eligibleForUnstake(ValidatorInfo storage validator) internal view returns (bool) {
977     ValidatorStatus status = validator.currentStatus;
978     if (status == ValidatorStatus.Staked) return true;
979
980     uint32 eligibleEpoch = validator.exitEpoch + 1; // ← overflows for PendingExit
981     if (status == ValidatorStatus.Exited && currentEpoch >= eligibleEpoch) {
982         return true;
983     }
984     return false;
985 }
```

 Copy Code

ConsensusRegistry.sol : `_beginExit` sets `exitEpoch` to `type(uint32).max`:

```
703 function _beginExit(ValidatorInfo storage validator) internal {
704     validator.currentStatus = ValidatorStatus.PendingExit;
705     validator.exitEpoch = PENDING_EPOCH; // type(uint32).max
706     emit ValidatorPendingExit(validator);
707 }
```

 Copy Code

ConsensusRegistry.sol : `unstake` calls `_eligibleForUnstake`:

```
554 ValidatorInfo storage validator = validators[validatorAddress];
555 if (!_eligibleForUnstake(validator)) revert IneligibleUnstake(validator);
```

 Copy Code

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (2.5)

Recommendation

Consider moving the `exitEpoch + 1` computation inside the Exited status branch so it only executes when the value is safe, ensuring all non-eligible statuses return false and let the caller revert with the expected `IneligibleUnstake` error.

Remediation Comment

SOLVED: The `Rayls` team fixed this finding in the specified commit by moving the `exitEpoch + 1` computation inside the `Exited` status check, so it only runs when `exitEpoch` holds a real epoch value. `PendingExit` validators now correctly receive `IneligibleUnstake` error instead of `Panic(0x11)`.

Remediation Hash

<https://github.com/raylsnetwork/axyl/commit/d8fe68a15143b82377ae7ef1c15ab531ae857425>

7.12 BLS IDENTITY POINT DETECTION UNREACHABLE DUE TO INPUT/CONSTANT SIZE MISMATCH

// LOW

Description

The `isInfinityPointG1()` and `isInfinityPointG2()` functions in `BlsG1.sol` compare the keccak256 hash of their input against the hash of the `G1_IDENTITY` / `G2_IDENTITY` constants, which are 128 and 256 bytes respectively. However, the callers `encodeG1PointForEIP2537()` and `encodeG2PointForEIP2537()` pass 96-byte and 192-byte uncompressed points. Since keccak256 of differently-sized byte arrays can never be equal, the identity early-return is dead code.

When a caller provides the 96-byte all-zero G1 identity point (or 192-byte G2 identity), the guard that should detect it and return a properly-formatted 128-byte (or 256-byte) EIP-2537 identity representation silently fails. Instead, the all-zero input proceeds through the `fieldElementToEIP2537Bytes` padding path, which produces a 128-byte output with 16-byte zero padding prepended to each 48-byte zero coordinate. While the resulting bytes are all zeros and happen to match the correct EIP-2537 identity encoding in this specific case, the guard's failure means the identity detection logic cannot be relied upon by other callers of `isInfinityPointG1()` or `isInfinityPointG2()` which may produce incorrect behavior depending on the context.

Code Location

`BlsG1.sol` : `encodeG1PointForEIP2537()` function validates the input length as 96 bytes, then passes it to `isInfinityPointG1()`

```
85 | function encodeG1PointForEIP2537(  
86 |     bytes memory uncompressedSignature  
87 | ) external pure returns (bytes memory) {  
88 |     if (uncompressedSignature.length != 96) {  
89 |         revert InvalidBLSProof();  
90 |     }  
91 |  
92 |     // if the provided signature is the identity, return early with 128 zero bytes  
93 |     bytes memory encoded = new bytes(EIP2537_G1_POINT_SIZE);  
94 |     if (isInfinityPointG1(uncompressedSignature)) return encoded; // ← never true  
95 |  
96 |     // otherwise, pad x and y coordinates with 16 leading zeroes  
97 |     fieldElementToEIP2537Bytes(uncompressedSignature, 0, encoded, 0);  
98 |     fieldElementToEIP2537Bytes(uncompressedSignature, 48, encoded, 64);  
99 |  
100 |     return encoded;  
101 | }
```

 Copy Code

The `isInfinityPointG1()` function compares the hash of its input against `G1_IDENTITY`, which is defined as a 128-byte zero array:

```
53 | bytes public constant G1_IDENTITY = new bytes(EIP2537_G1_POINT_SIZE); // 128 zero bytes
```

 Copy Code


```

153 | function isInfinityPointG1(
154 |     bytes memory uncompressedPointG1
155 | ) public pure returns (bool) {
156 |     return keccak256(uncompressedPointG1) == keccak256(G1_IDENTITY);
157 |     //      ^^^^^^^^^ 96 bytes                ^^^^^^^^^ 128 bytes → never equal
158 | }

```

BlsG1.sol : The same pattern applies to the G2 path, where **encodeG2PointForEIP2537()** passes a 192-byte input while G2_IDENTITY is 256 bytes

```

54 | bytes public constant G2_IDENTITY = new bytes(EIP2537_G2_POINT_SIZE); // 256 zero bytes

```

BVSS

AO:A/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (2.5)

Recommendation

Consider defining separate identity constants that match the actual input sizes used by each encoding function. The G1 encoder receives 96-byte uncompressed points, so a 96-byte zero constant should be used for the G1 identity check. Similarly, a 192-byte constant should be used for the G2 path.

Remediation Comment

SOLVED: The **Rayls** team fixed this finding in the specified commit by adding two new raw-size identity constants (96 and 192 bytes) and matching helper functions, then switching the broken call sites in **encodeG1PointForEIP2537** and **encodeG2PointForEIP2537** to use them so the identity early-return actually works.

Remediation Hash

<https://github.com/raylsnetwork/axyl/commit/6305f4e590d0f169fb5a6f8e029f9fc3ba05f210>

7.13 HARDCODED ERC-7201 STORAGE SLOT DOES NOT MATCH ITS DOCUMENTED NAMESPACE DERIVATION

// INFORMATIONAL

Description

The RLS_STORAGE_SLOT constant in RLS.sol is annotated as the ERC-7201 derivation of `rls.token.storage.v1`, but the hardcoded value `0x7bdaea41...` does not match the actual output of that computation (`0x9ceba13d...`). This mismatch between the `@custom:storage-location` annotation and the runtime slot creates a potential upgrade-safety hazard when tooling relies on the annotation to verify storage layout compatibility.

BVSS

AO:A/AC:L/AX:H/R:N/S:U/C:N/A:N/I:M/D:N/Y:N (1.6)

Recommendation

Consider updating RLS_STORAGE_SLOT to the correct derivation of the documented namespace, or updating the namespace string and annotation to match the existing hardcoded slot.

Remediation Comment

SOLVED: The `Rayls` team solved this finding in the specified commit by replacing `RLS.sol` with a v2 implementation that removes the ERC-7201 namespaced storage pattern entirely, eliminating the incorrect `RLS_STORAGE_SLOT` constant and its mismatched annotation in favour of a conventional `__gap` array.

Remediation Hash

<https://github.com/raylsnetwork/axyl/commit/47dd8318733c69ec34f521927c95908759f4ddc7>

7.14 RING BUFFER EXPANSION CAUSES PERMANENT ATTESTATION DOS

// INFORMATIONAL

Description

The `setBufferSize()` function in `GitAttestationRegistry` fails to pad the storage array when expanding to a larger size, leaving `ringBuffer.length` shorter than the updated `bufferSize` counter. The next call to `attestGitCommitHash()` writes to an out-of-bounds index, causing a Solidity 0.8 array-access panic that permanently blocks all future attestations.

After any buffer expansion, both write and read operations on the registry permanently revert with an array out-of-bounds panic. No new git commit hashes can be attested, and querying existing attestations beyond the old buffer size also fails. The contract has no self-repair mechanism, calling `setBufferSize()` again to shrink back does not help because head already points beyond `ringBuffer.length`, causing the copy loop's `ringBuffer[(start + i) % bufferSize]` to panic as well. The contract is effectively bricked until redeployed.

Code Location

`GitAttestationRegistry.sol` : The `setBufferSize()` function copies existing records into a memory array and re-pushes them to storage, but the push loop only iterates `itemsToCopy` times (equal to the old buffer size when expanding), not `newSize` times

 Copy Code

```
60 function setBufferSize(  
61     uint8 newSize  
62 ) external onlyRole(DEFAULT_ADMIN_ROLE) {  
63     require(newSize > 0, "Buffer size cannot be 0");  
64  
65     GitCommitHashRecord[] memory newBuffer = new GitCommitHashRecord[](  
66         newSize  
67     );  
68     uint8 itemsToCopy = bufferSize < newSize ? bufferSize : newSize;  
69     // calculate start index of oldest element in ring buffer  
70     uint8 start = (head + (bufferSize - itemsToCopy)) % bufferSize;  
71  
72     for (uint8 i; i < itemsToCopy; ++i) {  
73         newBuffer[i] = ringBuffer[(start + i) % bufferSize];  
74     }  
75  
76     // solidity does not support writing memory arrays directly to storage  
77     delete ringBuffer;  
78     for (uint8 i; i < itemsToCopy; ++i) {           // ← only pushes oldSize items  
79         ringBuffer.push(newBuffer[i]);  
80     }  
81  
82     bufferSize = newSize;                           // ← set to newSize (larger)  
83     head = itemsToCopy % newSize;                   // ← points beyond ringBuffer.length  
84  
85     emit BufferSizeChanged(newSize);  
86 }
```

`GitAttestationRegistry.sol` : The subsequent call to `attestGitCommitHash()` writes to `ringBuffer[head]`, which is now an out-of-bounds index.

```

39 function attestGitCommitHash(
40     bytes20 gitCommitHash,
41     bool ciPassed
42 ) external onlyRole(MAINTAINER_ROLE) {
43     ringBuffer[head] = GitCommitHashRecord(gitCommitHash, ciPassed); // ← PANIC
44     head = (head + 1) % bufferSize;
45
46     emit GitHashAttested(gitCommitHash, ciPassed);
47 }

```

`GitAttestationRegistry.sol` : Additionally, `gitCommitHashAttested()` iterates up to `bufferSize`, which also exceeds `ringBuffer.length` and panics

```

49 function gitCommitHashAttested(
50     bytes20 gitCommitHash
51 ) external view returns (bool) {
52     for (uint8 i = 0; i < bufferSize; i++) {
53         if (ringBuffer[i].gitCommitHash == gitCommitHash) { // ← PANIC when i >= ringBuffer.l
54             return ringBuffer[i].ciPassed;
55         }
56     }
57     return false;
58 }

```

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:H/I:N/D:N/Y:N (1.5)

Recommendation

Consider padding the storage array with empty records after copying the existing data, so that `ringBuffer.length` matches `bufferSize`. The constructor already uses this pattern, pushing empty `GitCommitHashRecord` entries to fill the array to `bufferSize_`.

Remediation Comment

SOLVED: The `Rayls` team resolved this finding in the specified commit by deleting `GitAttestationRegistry.sol` and its associated test file entirely as part of a broader unused-contract cleanup, eliminating the vulnerable code from the codebase.

Remediation Hash

<https://github.com/raylsnetwork/axyl/commit/499508022659613e1426c24176f54da279bc56f8>

7.15 _EJECT() SWAP-AND-POP BREAKS COMMITTEE SORT ORDER AFTER VALIDATOR BURN

// INFORMATIONAL

Description

`ConsensusRegistry._eject()` uses a swap-and-pop pattern to remove a validator from a committee array, which does not preserve ascending address order. When `_consensusBurn()` triggers committee ejection after a validator's stake is fully slashed, previously sorted committees may become unsorted.

Committees written by `concludeEpoch()` are guaranteed to be sorted in ascending address order. Protocol component, both on-chain (e.g., `_isCommitteeMember()` lookups, committee deduplication) and off-chain (e.g., the Rust consensus layer's committee comparison logic, deterministic message ordering based on committee position) may rely on this invariant.

BVSS

AO:S/AC:L/AX:L/R:N/S:U/C:N/A:N/I:L/D:N/Y:N (0.5)

Recommendation

Consider replacing the swap-and-pop with a sort-preserving shift-left deletion that maintains ascending order.

Remediation Comment

SOLVED: The `Rayls` team fixed this finding in the specified commit by replacing the swap-and-pop pattern in `_eject` with a shift-left deletion that preserves the committee's ascending order across ejections.

Remediation Hash

<https://github.com/raylsnetwork/axyl/commit/6e4c622f8ac9a32ce7d33ae5b228431604734005>

Halborn strongly recommends conducting a follow-up assessment of the project either within six months or immediately following any material changes to the codebase, whichever comes first. This approach is crucial for maintaining the project's integrity and addressing potential vulnerabilities introduced by code modifications.